

%rax	%eax	%ax	%al	Return Value (Caller saved)	%r8	%r8d	%r8w	%r8b	5 th Argument (Caller saved)
%rbx	%ebx	%bx	%bl	Caller saved	%r9	%r9d	%r9w	%r9b	6 th Argument (Caller saved)
%rcx	%ecx	%cx	%cl	4 th Argument (Caller saved)	%r10	%r10d	%r10w	%r10b	Caller saved
%rdx	%edx	%dx	%dl	3 rd Argument (Caller saved)	%r11	%r11d	%r11w	%r11b	Caller saved
%rsi	%esi	%si	%sil	2 nd Argument (Caller saved)	%r12	%r12d	%r12w	%r12b	Caller saved
%rdi	%edi	%di	%dil	1 st Argument (Caller saved)	%r13	%r13d	%r13w	%r13b	Caller saved
%rsp	%esp	%sp	%spl	Stack pointer	%r14	%r14d	%r14w	%r14b	Caller saved
%rbp	%ebp	%bp	%bpl	Caller saved	%r15	%r15d	%r15w	%r15b	Caller saved

do-while	C (normal)	C (goto)
do Body while (Test);	do Body while (Test); goto loop;	loop: Body if (Test) goto loop;
while (Test) Body	"jump to middle" Body	"guarded to" (do-while) if (!Test) goto done; loop: Body if (Test) goto loop; done:

for (Init; Test; Update) Body	same as: Init while (Test) Body Update	NB: 'break' = jump to done 'continue' = jump to Update
---	---	--

switch (x) case n_1 : Body1 break case n_2 : Body2 (etc.) default: Body0	compact: $n_1, \dots, n_k = 1, \dots, k$ (or other small values) jump table .section .rodata .align 8 .L4: .quad .L3 # x = 0 .quad .L8 # x = 1 .quad .L5 # x = 2 .quad .L5 # x = 3 (etc.) switch code movq %6, %rdi # x ∈ [0,6] ja .L8 # default case jmp *.L4(,%rdi,8) # table[x]	NB: binary search (using an even search tree) then set jumps accordingly (until you find/don't find value)
---	--	---

int* pc, c; c = 5; pc = &c;	Here, 5 is assigned to the c variable. And, the address of c is assigned to the pc pointer
--	---

%rax	Register	Expression	Type	Formula	Assembly Code
0x210	Abs. Memory	A+3	short*	$x_A + 6$	leaq 6(%rdi), %rax
%0x210	Immediate	A[5]	short	$M[x_A + 10]$	movw 10(%rdi), %ax
(%rcx)	Memory	A[4*i+2]	short	$M[x_A + 8 * i + 4]$	movw 4(%rdi,%rsi,8), %ax

imulq src R[%rdx]:R[%rax] ← src × R[%rax]	Signed multiplication
mulq src R[%rdx]:R[%rax] ← src × R[%rax]	Unsigned multiplication
idivq src R[%rdx] ← R[%rdx]:R[%rax] mod src; R[%rax] ← R[%rdx]:R[%rax] ÷ src	Signed divide
divq src R[%rdx] ← R[%rdx]:R[%rax] mod src; R[%rax] ← R[%rdx]:R[%rax] ÷ src	Unsigned divide
cqto R[%rdx]:R[%rax] ← signExtend(R[%rax])	Convert to octal word

(total) memory size = 2^M C = cache size = $S \times N \times B$ B = 2^0 = block size S = 2^I = # cache sets = $C/B/N$ N = associativity (# lines per set) # lines (or blocks) = C/B M = address size	$T = M - (I + O)$ User for tag comparison Tag (T) Decreasing associativity ←	$I = \log_2 C/B/N$ Selects the set Index (I) Increasing associativity →	$O = \log_2 B$ Selects the byte from block Offset (O) Fully associative (only one set) $N = C/B, I = 0$
--	--	---	--

Temporal locality	Recently referenced ⇒ likely to be referenced again soon
Spatial locality	Items w/ nearby addresses ⇒ likely referenced close in time

arrays of arrays: memory layout (of e.g. int)

Nested (each row e.g. 5 cols)	$\text{Mem}[x_A + 20 * i + 4 * j]$
Multi-level (rows of diff size)	$\text{Mem}[\text{Mem}[x_A + 8 * i] + 4 * j]$

structs: memory layout

address of structure **and** structure length must be multiples of $K_{\max}$

data type is size K bytes $\Rightarrow$ address must be multiple of K

valid? + match? : yes = hit

V

tag

0

1

2

3

Memory cannot mem-mem transfer w/ single instr.
mov Src, Dest Dest = Src int x = y
lea Src, Dest Dest = &Src x = x + k*y
movzSD Src, RegDest w/ zero extension
movsSD Src, RegDest w/ sign extension
from smaller src (size: suffix S) - mem or reg (not imm) to larger dest (size: suffix D) – register

opc.	args	operation (in C)	description
addl	src,dest	dest = dest + src	Addition
subl	src,dest	dest = dest - src	Subtraction
imull	src,dest	dest = dest * src	Multiplication
sall	src,dest	dest = dest << src	Shift arith. Left
sarl	src,dest	dest = dest >> src	Shift arith. Right
xorl	src,dest	dest = dest ^ src	Bitwise xor
andl	src,dest	dest = dest & src	Bitwise and
orl	src,dest	dest = dest src	Bitwise or
incl	dest	dest = dest + 1	Inc. by 1
decl	dest	dest = dest - 1	Dec. by 1
negl	dest	dest = -dest	Negate
notl	dest	dest = ~dest	Bitwise not

Computer	long/pointer	Long long
32 bit linux	4 bits (max alignment of 4)	8 bits
64 bit linux	8 bits (max alignment of 8)	8 bits

CF: carry (unsigned)	ZF: zero	SF: sign (signed)	OF: overflow (signed)
cmp s2, s1	s1 - s2 (discard result: set flags)	Condition Codes (CCs) & conditions	
test s2, s1	s1 & s2 (discard result: set ZF, SF)		
jmp target	goto (uncond.) direct		
jmp *operand	goto (uncond.) indirect		
j/set args condition description	cmp b,a test b,a		
je dst ZF Equal / Zero	a == b a&b == 0		
jne dst ~ZF Not equal / Not zero	a != b a&b != 0		
js dst SF Negative		a&b < 0	
jns dst ~SF Nonnegative		a&b >= 0	
jg dst ~(SF^OF)&~ZF Greater	a > b	a&b > 0	
jge dst ~(SF^OF) Greater or equal	a >= b	a&b >= 0	
jle dst (SF^OF) ZF Less	a < b	a&b < 0	
jle dst (SF^OF) ZF Less or equal	a <= b	a&b <= 0	
ja dst ~CF&~ZF Above (unsigned)	a > b		
jb dst CF Below (unsigned)	a < b		

cmovbe src,dest if (a == b) dest = src	Condit. mov: equal/zero
^both branches are evaluated	
if (x > y) res = x-y; else res = y-x;	subq %rsi, %rax # res = x-y movq %rsi, %rdx # eval = y-x subq %rsi, %rdi # x:y cmovbe %rdx, %rax # if <=, res = eval

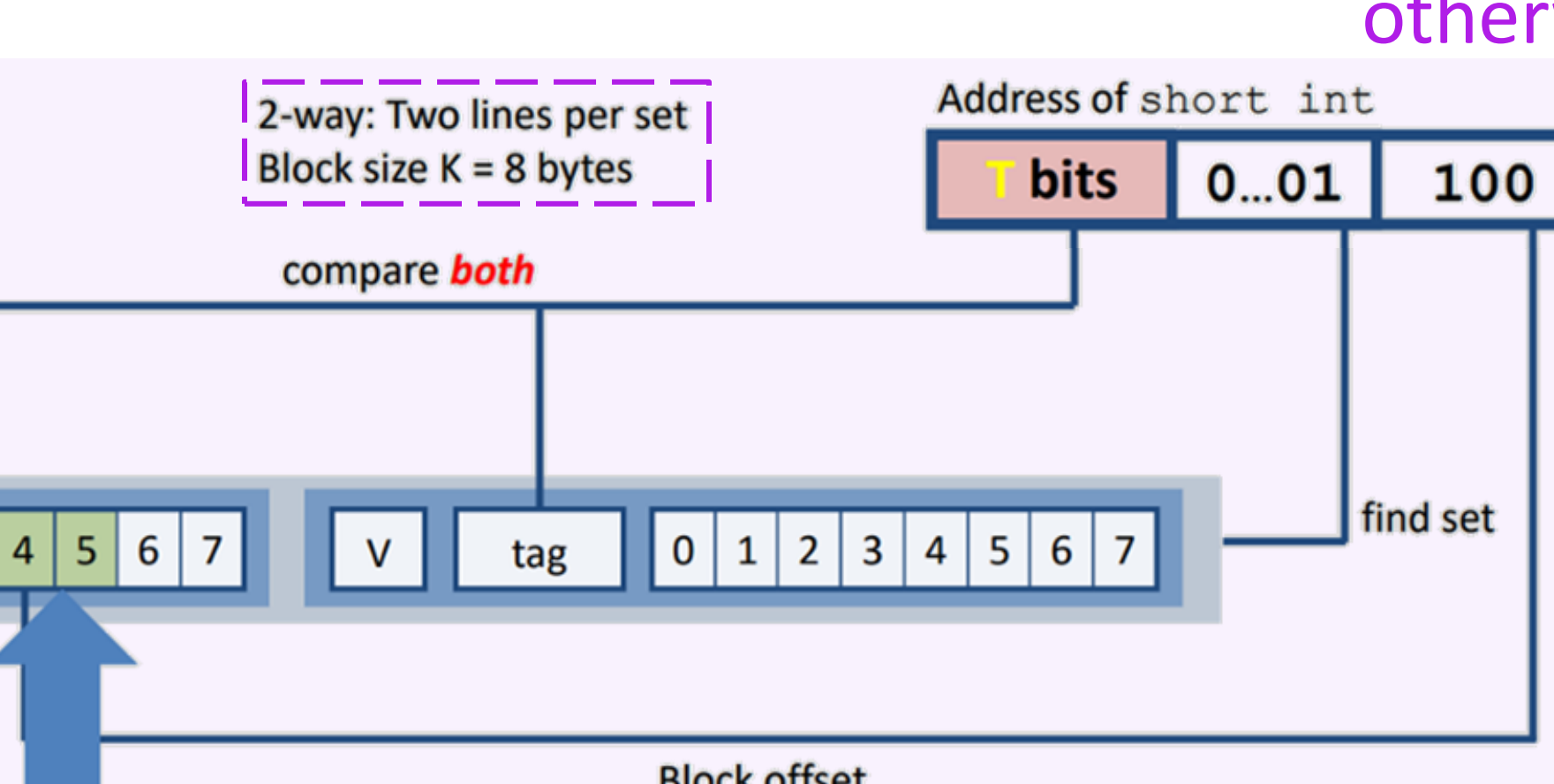
Alignment Requirement: dictated by the largest alignment requirement inside, which is 8 for long and pointers

Twos' complement (signed):
range $[-2^{n-1}, \dots, 2^{n-1} - 1]$
overflow: $A, B > 0, A + B \leq 0$
 $A, B < 0, A + B \leq 0$

The diagram illustrates a memory layout, likely a stack frame. It features a large trapezoidal shape representing a memory block. Inside this block, there is a smaller rectangle labeled '32', which likely represents a return address or a constant value. Below this, there is a box labeled 'pr', which likely represents a pointer variable. An arrow points from the top of the trapezoid to the '32' box, and another arrow points from the 'pr' box to the bottom of the trapezoid, indicating a flow or relationship between the two components.

MR	miss rate	frac. of mem. refs not found in cache (= 1 - hit rate)
HT	hit time	Time(block in cache?) + Time(block → processor (from cache))
MP	miss penalty	additional time req. due to miss
AMAT	avg. mem. access time	AMAT = HT + MR × MP

[array elems] Access pattern: stride n
replacement policies: LRU, NMRU



opc.	args	operation	description
pushq	src	(fetch src) %rsp = %rsp - 8 *(%rsp) = src	add src to top of stack
pop	dst	(read oprnd @ %rsp) %rsp = %rsp + 8 dst = oprnd	pop operand from top of stack to dst
call	label	push (return addr.) jmp label	go to label
ret		pop (return addr.) jmp (return addr.)	return from a call

Stack used if: [avoid if poss.! slower!!]

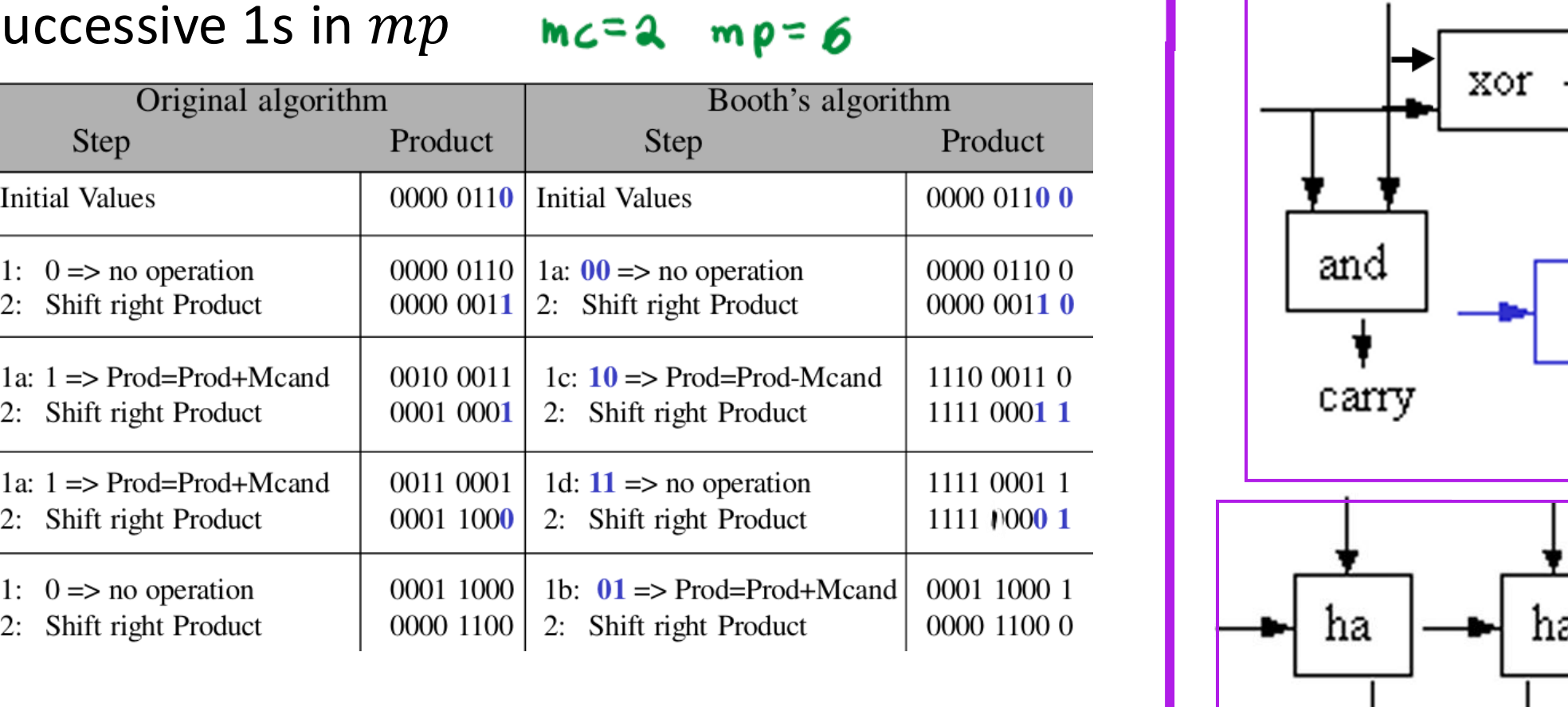
- not enough registers for all data
- we use '&' for the variable at some pt
- array or struct used as local vars

Stack w/w	bottom top	Local variables; procedure context
Dynamic data (heap) w/w		Managed by programmer
Uninitialised data w/w		Vars allocated w/ new or malloc
Initialised data w/w		Static vars
Instructions / text w/w		Literals
		Initialized when process starts

Booth's algorithm (signed) check the 2LSB of pr (product register, stores result)

Geometric series
(for a k-length string of 1s)
 $m + 2m + \dots + 2^{k-1}m$
 $= -m + 2^k m$
 $= -m + (m << k)$

based on successive 1s in mp
 $mc = 2, mp = 6$

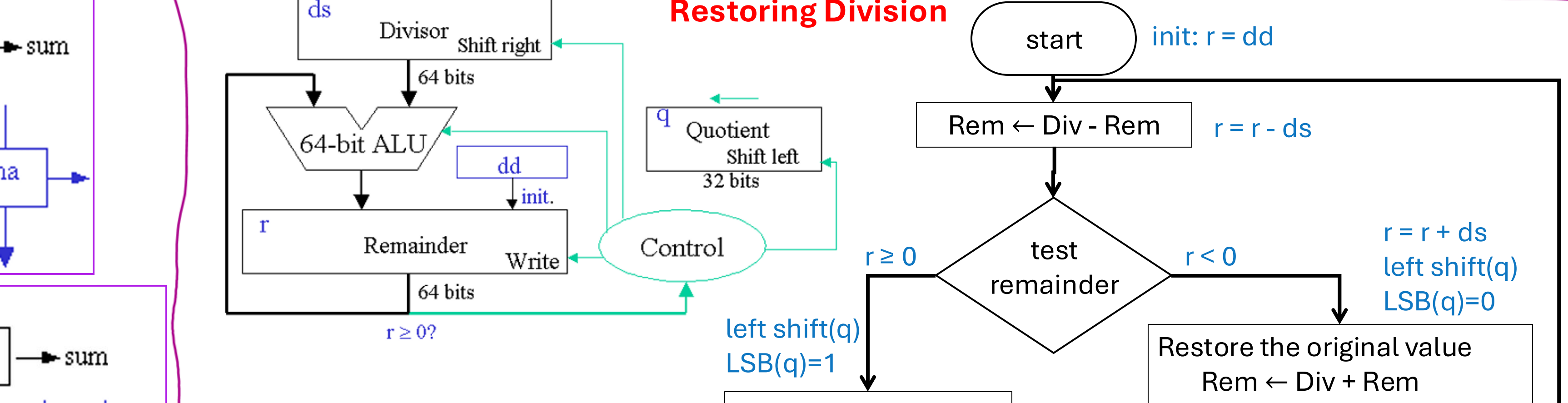
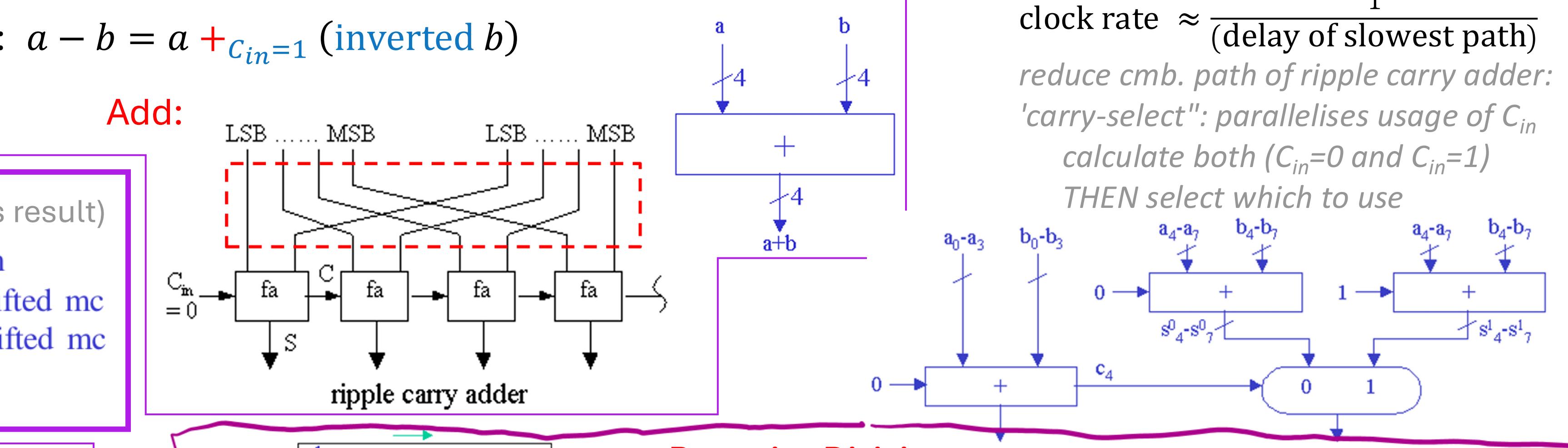
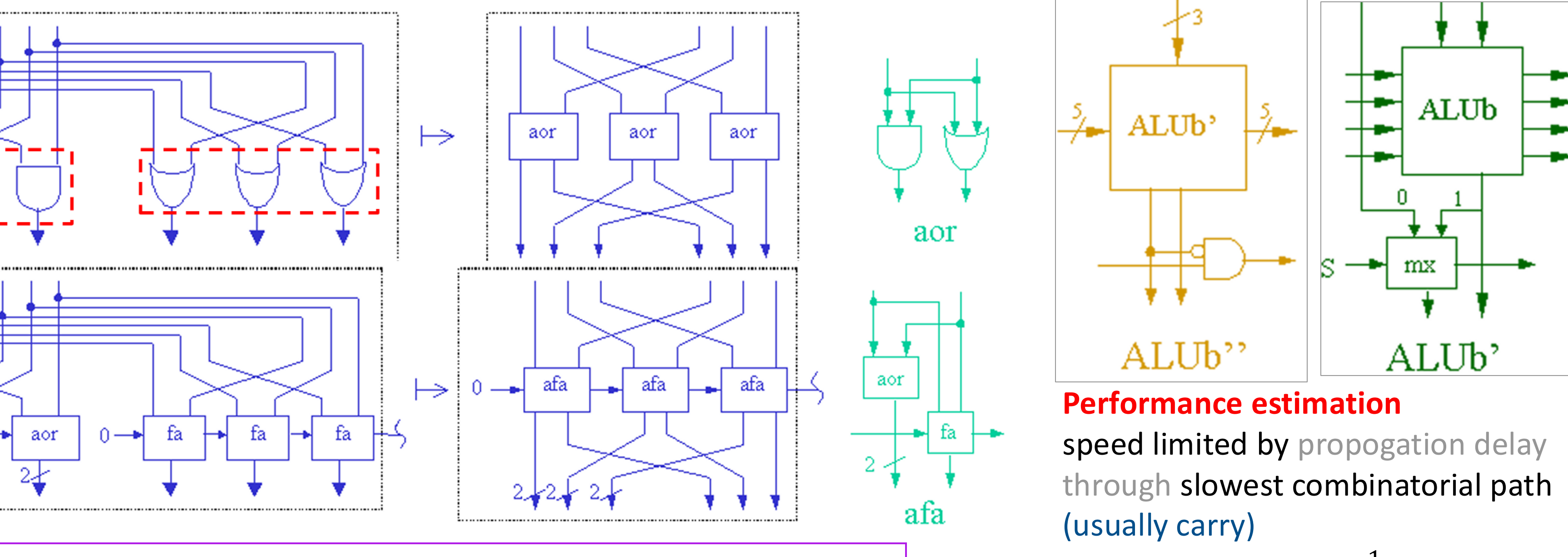
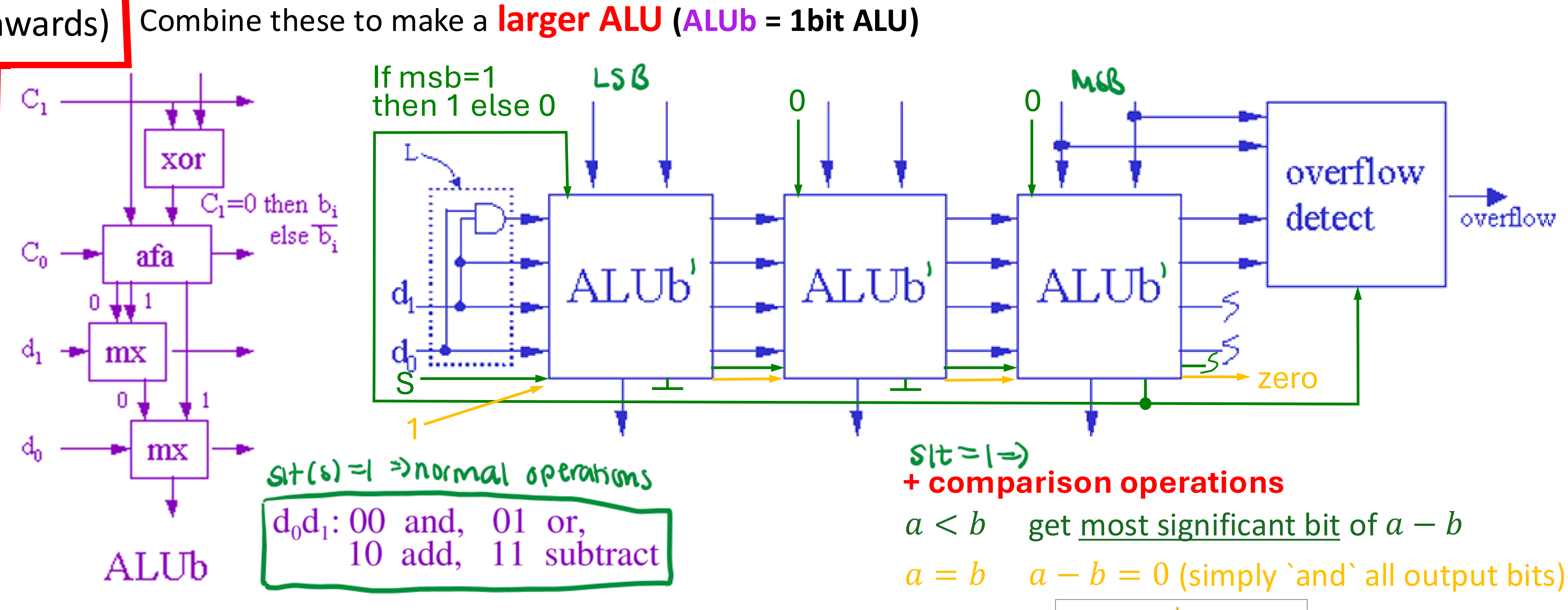


Step	Divisor	Remainder
0	Initial Values	0010 0000 0000 0111
1	1: Rem = Rem - Div 2a: Rem < 0 => +Div, slt Q, Q0=0 3: Shift Div right	0010 0000 0110 1110
2	1: Rem = Rem - Div 2b: Rem < 0 => +Div, slt Q, Q0=0 3: Shift Div right	0010 0000 0000 1000
3	1: Rem = Rem - Div 2a: Rem < 0 => +Div, slt Q, Q0=0 3: Shift Div right	0010 0000 0000 0100
4	1: Rem = Rem - Div 2a: Rem < 0 => +Div, slt Q, Q0=0 3: Shift Div right	0010 0000 0000 0010
5	1: Rem = Rem - Div 2a: Rem < 0 => +Div, slt Q, Q0=0 3: Shift Div right	0010 0000 0000 0001

Step	Divisor	Remainder
0	Initial Values	0010 0000 0000 0111
1	1: Shift left, R0=0 2a: Rem < 0 => +Div, slt Q, Q0=0 3: Shift Div right	0010 0000 0110 1110
2	1: Shift left, R0=0 2b: Rem < 0 => +Div, slt Q, Q0=0 3: Shift Div right	0010 0000 0000 1000
3	1: Shift left, R0=0 2a: Rem < 0 => +Div, slt Q, Q0=0 3: Shift Div right	0010 0000 0000 0100
4	1: Shift left, R0=0 2a: Rem < 0 => +Div, slt Q, Q0=0 3: Shift Div right	0010 0000 0000 0010
5	1: Shift left, R0=0 2a: Rem < 0 => +Div, slt Q, Q0=0 3: Shift Div right	0010 0000 0000 0001

Types of misses

- Compulsory (cold) miss:** first access to block
- Conflict miss:** cache large enough, but multiple data objects map to same slot. direct-mapped ⇒ many conflict misses fully-associative ⇒ no conflict misses
- Capacity miss:** set of active cache blocks (the working set) larger than the cache



Iteration	Step	Quotient	Divisor	Remainder
0	Initial Values	0000	0010 0000	0000 0111
1	1: Rem = Rem - Div 2b: Rem < 0 => +Div, slt Q, Q0=0 3: Shift Div right	0000	0010 0000	0000 0111
2	1: Rem = Rem - Div 2b: Rem < 0 => +Div, slt Q, Q0=0 3: Shift Div right	0000	0001 0000	0000 0111
3	1: Rem = Rem - Div 2b: Rem < 0 => +Div, slt Q, Q0=0 3: Shift Div right	0000	0000 1000	0000 0111
4	1: Rem = Rem - Div 2a: Rem < 0 => +Div, slt Q, Q0=0 3: Shift Div right	0001	0000 0100	0000 0111
5	1: Rem = Rem - Div 2a: Rem < 0 => +Div, slt Q, Q0=0 3: Shift Div right	0001	0000 0010	0000 0001

Matrix multiplication has inherent temporal locality

- Input data: $3n^2$, computation $2n^3$
- Every array element used $O(n)$ times!
- But program has to be written properly

Blocking optimization only works if the blocks fit in the cache

- Suggests larger possible block size up to limit $3n^2 \leq C$

CPI is an avg. of all the instructions executed in a program. CPI provides a way of comparing two different implementations of the same ISA, since the number of instructions executed for a program is the same

field values

control signal assignment

ALU control:

Add

F_nCode

Subtract

ALUOp = 00

ALUOp = 10

ALUOp = 01

(mux control)

PC

A

ALUSrcA = 0

ALUSrcA = 1

← PC is first operand

← Reg. A is first operand

SRC1:

PC

ALUSrcA = 0

SRC2:

B

ALUSrcB = 00

4

ALUSrcB = 01

Extend

ALUSrcB = 10

ExtShift

ALUSrcB = 11

Memory:

Read PC

MemRead, IorD = 0, IRWrite

Read ALU

MemRead, IorD = 1

Write ALU

MemWrite, IorD = 1

← address in PC

← address from ALUOut

Reg Control:

Read

A = Reg[IR₂₅₋₂₁], B = Reg[IR₂₀₋₁₆]

Write ALU

RegWrite, MemtoReg = 0, RegDst = 1

Write MDR

RegWrite, MemtoReg = 1, RegDst = 0

PCWrite Control:

ALU

PC source = 00, PC write

ALUoutcond

PC WriteCond, PC source = 01

Jump addr

PC write, PC source = 10

Exception

PC write, PC source = 11

constant value (pointer to code)

Cause:

Int

IntCause = 1, CauseWrite

Op

IntCause = 0, CauseWrite

Sequencing:

seq

next state = current state + 1

fetch

goto label fetch, back to state 0

dispatch 1

next state from dispatch 1

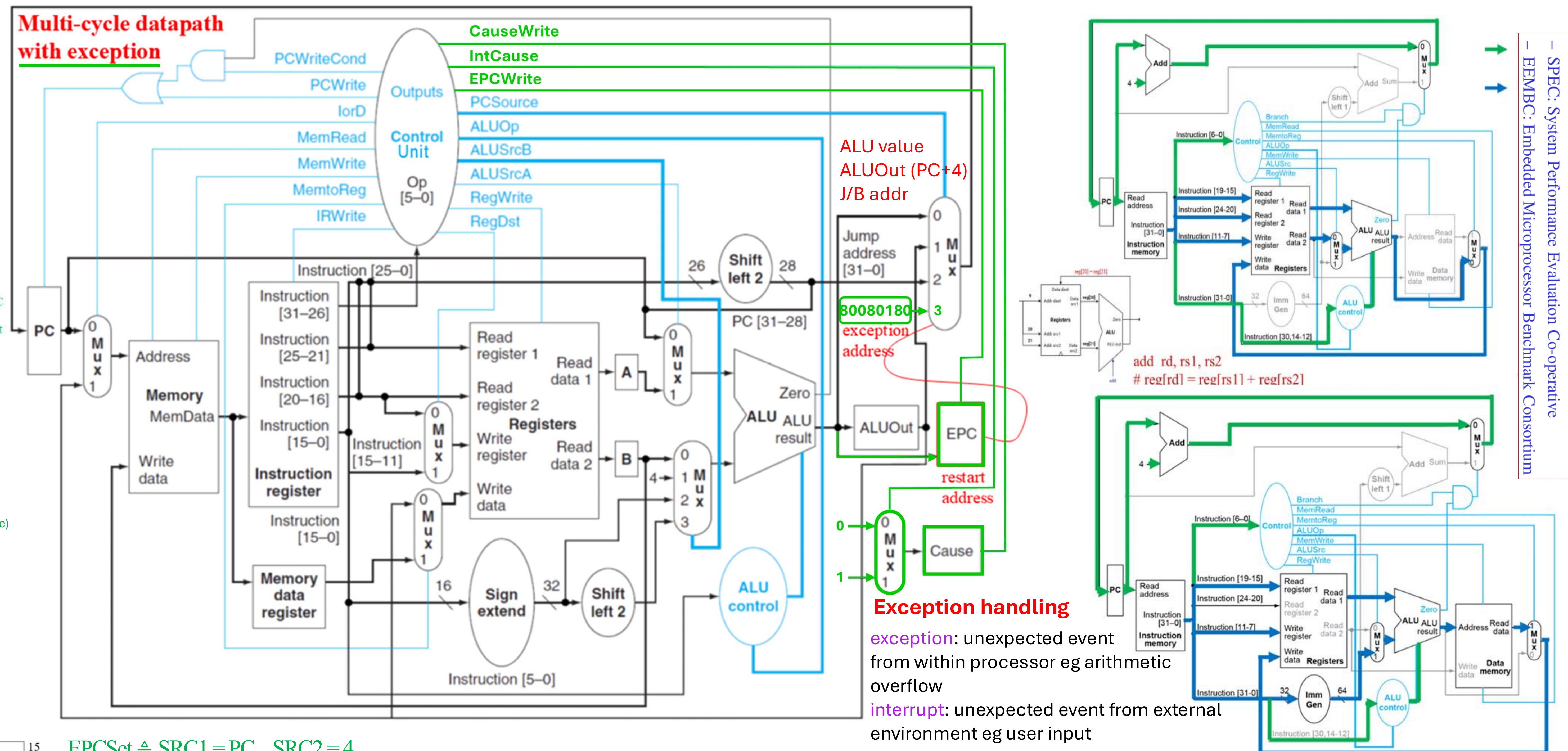
dispatch 2

next state from dispatch 2

R/I-wb

goto state 4 from 2

Via jumptable: opcode-state



ALU: ALUOp

control 4 mux: RegDst, ALUSrc, MemtoReg, PCSrc

storage: Register Write, Data Memory Read/Write

	instruction fetch	read register	ALU operation	load/store data	write register
R type	✓	✓	✓		✓
load		✓		✓	✓
store		✓		✓	
branch		✓	✓		
jump			✓		✓

Branch (I)
beq \$5, \$6, L
If reg[5]=reg[6]:
jump by offset L

direct FSM hard-wired control unit

- ROM size: 11 bit (7-bit opcode, 4-bit state) x 19 (15-bit control, 4-bit state)
- total size: 2¹¹x19 = 38.9K bits

microsequencer control unit

ROM-based logic: 4-bit state x 18 control signals

2 dispatch ROMs, each with 7-bit input and 4-bit output

total size: 2⁴x18 + 2(2⁷x4) = 1.31K bits

number of cycles for program P = number of instr. for P x CPI

execution time for P = clock cycle for P x number of cycles for P

average exe. time for P1, P2...Pn (assume equal workload) = 1/n (exe. time for P1 + ... + exe. time for Pn)

(CPI = avg. clock cycles per instruction)

exe. time = instr. count x CPI x cycle time

Microsequencer

sequentialise microinstruction to maximise encoding

e.g. k-state horizontal design with 16 outputs (m=16)

replace each microinstruction by 3 encoded microinstructions with 4 outputs (n=4) and a 4-to-16 decoder

ROM saving = 16k - 4(3k) = 4k bits

DR1

Input	Output
R-type opcode	0010 (state 2)
I-type opcode	0011 (state 3)
lw opcode	0101 (state 5)
S-type opcode	1001 (state 9)
B-type opcode	1010 (state 10)

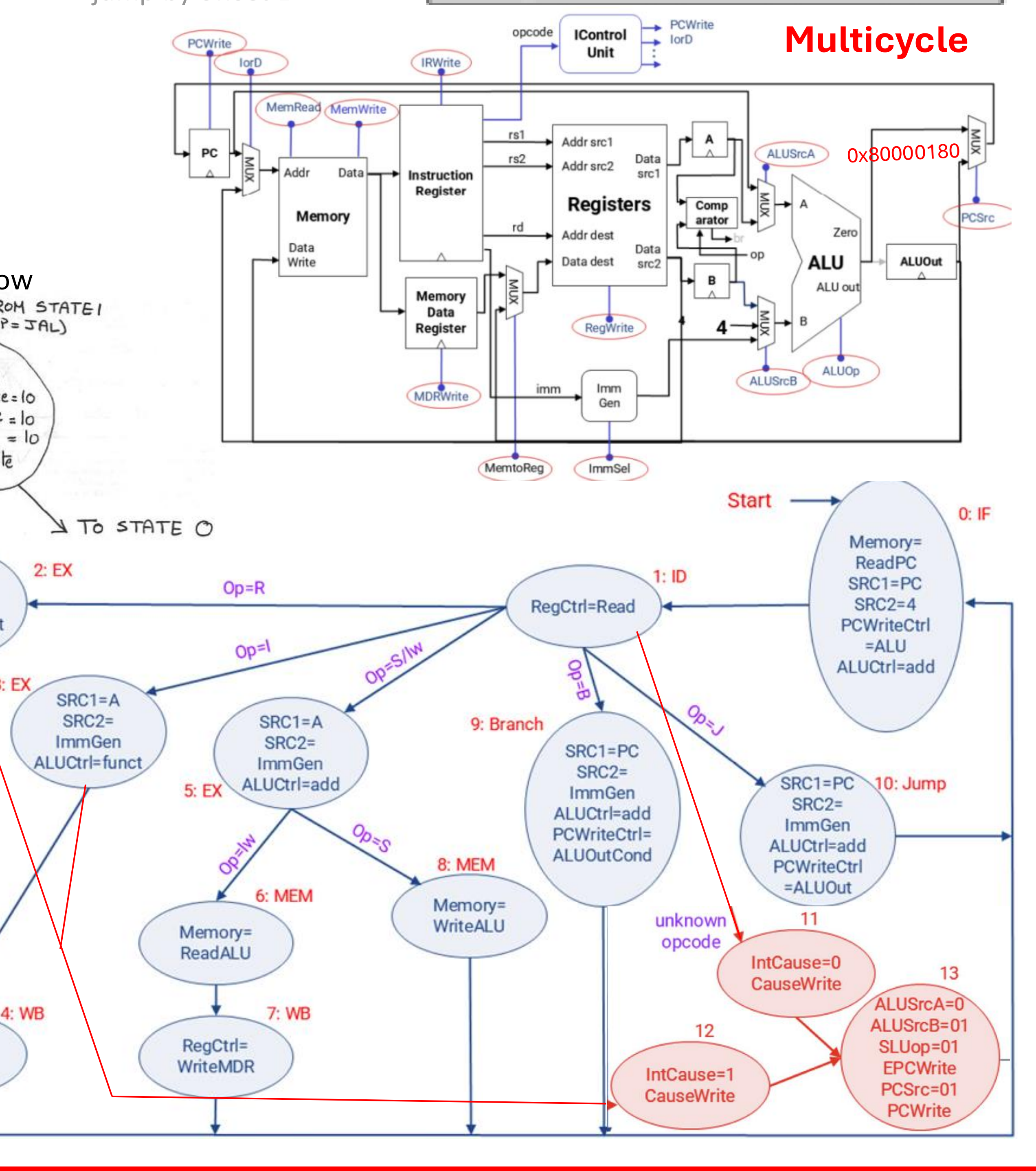
ROM size: 2^a x d

Impls. combinatorial logic for control unit

Register Transfer Language

ld rd, imm(rs1) # reg[rd] = mem[reg[rs1] + sgx(imm[11:0])]

Jump and link: R31 = PC; PC = PC[31-28] || (IR[25-0] << 2)



Microprogram: field assignment table

State	Label	ALU Control	SRC 1	SRC 2	Memory	Reg. Control	PC write Control	Sequencing
0	Fetch	Add	PC	4	ReadPC		ALU	Seq
1		Add	PC	ExtShift		Read		Dispatch 1
2	Mem1	Add	A	Extend				Dispatch 2
3	LW2				ReadALU			Seq
4						WriteMDR		Fetch
5	SW2						WriteMDR	Fetch
6	Rformat1	Fn.code	A	B	WriteALU			Seq
7						WriteALU		Fetch
8	BEQ1	subt	A	B			ALUoutcond	Fetch
9	JUMP1						Jump addr	Fetch

horizontal: 1-bit of control logic per control output bit (i.e. n = m)

[|ROM|] |IN| = |opcode| + |state|, |OUT| = |control| + |state|

+ exploits parallelism of ops in DP. + minimal control overhead

- requires larger ROM

vertical: encode comb. logic s.t. n < m. decoder gives signals

+ one operation at a time: easier to understand & use

- slower: decoding delay, sequentialism of instructions

⇒ encode only non-time-critical parts

disadv: hard-wired control faster, cheaper memory: may not require microseq for RISC, advance in design tools for optimising hard-wired control, bottleneck if microcode stored in slow memory

Best of both worlds: most instr: exec w/ hard-wired control, complex instr: invoke microcode

Architectures:	C=A+B	Other	Pros	Cons
Stack	push A push B add pop C	push b, neg, push a, add, dup, pop C	simple evaluation dense code	less flexible: no random access slow if stack is in memory
accumulator	load A add B store C	Load b, neg, add a, store c	min. internal store short instr.	freq. memory access → slow
Register/load/store	load \$1 A load \$2 B add \$2, \$1, \$2 store \$2 C	Pros reduce memory traffic – has enough reg, so only read/write to each memory location once	general model fast register access compiler friendly improve code density	name all operands long instructions/large code size
Memory-memory	add C A B		fewest instructions	Largest number of data accesses Largest instr size

number of cycles for program P = number of instr. for P x CPI

execution time for P = clock cycle for P x number of cycles for P

average exe. time for P1, P2...Pn (assume equal workload) = 1/n (exe. time for P1 + ... + exe. time for Pn)

(CPI = avg. clock cycles per instruction)

exe. time = instr. count x CPI x cycle time

Amdahl's law

Performance

T_{new} is such that (a fraction) α of code runtime of T_{old} is faster by β 'times']

T_{new} = αT_{old} / β + (1 - α)T_{old}

CISC Complex Instruction Set Computers

- dense code, simple compiler
- powerful instruction set, variable format

RISC Reduced Instruction Set Computers

- simple instructions, fixed format, optimising compiler
- speed, low development cost, adapt to new technology

Addressing modes

register addressing: data in registers

immediate addressing: data in instruction itself (I-type)

base addressing: data in memory, e.g. load/store instructions

Register: arithmetic / comparison / logical

Immediate: memory access / conditional branches / arithmetic w/ constants

Jumps: unconditional jumps (other jumps can be R or I)

Comparing performance of M1, M2

consider exe. time for M1

depends on particular programs and data / implementation / etc.

e.g. comparison

SPECint92: 21 (4.2 times slower)

SPECfp92: 15 (6.5 times slower)

cost: \$233 (4.7 times cheaper)

cost effectiveness?

MIPS is 4.2 times faster for integers

6.5 times faster for floats

MIPS is less effective for integers but more effective for floats

How do things affect (different aspects of) performance?

Algorithm

Language

Compiler

Instruction Set

Organisation

Technology

Instr. Count

(clearly)

statements translated to processor instr.

(clearly: affects translation)

instructions needed for a function

(clearly)

(clearly)

CPI

may favour faster/slower instr.

more/fewer indirect calls (level of abstraction)

(clearly: affects translation)

cost of each instruction

Clock Rate

MIPS

32 registers (\$0..\$31) - each 32 bits (\$0 = 0 const.)

Minimises memory access ("register-register" or "load-store" architecture)

Most instructions are register only (fast)

Special memory access instructions (often multicycle)

Instruction format

Usually destination comes first (e.g. add Rdest R1 R2)

	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits
R	opcode	source 1	source 2	dest.	shift amt	fn code
I	opcode	source 1	source 2 / dest.	address / data		
J	opcode	address				